

乱数を用いたプログラム

釧路工業高等専門学校情報工学科5年・情報工学実験II

氏名：クリスティアン・ハルジュノ

出席番号：21

学籍番号：234071

提出日：2025年7月21日

目次

1	初めに	1
2	背景	1
2.1	5つのシミュレーションを紹介	1
2.1.1	課題1: 7つのサイコロを振る	1
2.1.2	課題2: 半径7の円の面積	2
2.1.3	課題3: 異身長男女を一行に並べる	3
2.1.4	課題4: ジャンケンの割合	5
2.1.5	課題5: ビンゴのシミュレーション	6
2.2	シミュレーションの役割	9
2.3	モンテカルロシミュレーション	9
3	実験方法	10
3.1	ハードウェアと環境	10
3.2	ランダム関数	11
3.3	モジュロによるランダム関数のバイアス	13
3.4	実行数	15
4	実験結果	15

1 初めに

統計の世界において、ある事象が起こる「確率」の計算は $\frac{\text{条件を満たす場合の数}}{\text{全体の事象数}}$ という単純な式で表される。たとえば、コインを投げて表が出る確率は $\frac{1}{2}$ であり、これはコインに 2 面が存在するためである。しかし、実際にコインを 10 回投げてみると、表が出る割合は偏って見えることがあり、 $1/3$ や $1/2$ といった結果になる場合もある。ある意味で、確率には運による偏りの要素が依然として存在すると言える。このため、シミュレーションを多数回繰り返すことによって、「運」や「ノイズ」の影響を最小限に抑え、より正確な結果を得ることが重要である。

このレポートでは、乱数を用いたプログラムの実装とその結果の分析を行う。具体的には、コイン投げやサイコロの目の出方など、様々なランダムな事象をシミュレートし、それらの結果を統計的に解析することで、理論的な確率と実際の結果との関係を明らかにすることを目的とする。

2 背景

2.1 5つのシミュレーションを紹介

2.1.1 課題 1: 7つのサイコロを振る

本シミュレーションでは、7 個の 6 面サイコロを投げ、すべてのサイコロが同じ目を出す確率を求める。また、サイコロの個数を変化させながらシミュレーションを実行し、その結果を比較する。



図 1: 7つの同じ面のサイコロ

理論的には、一つの均質なサイコロにおいて各面が出る確率は正確に $\frac{1}{6}$ である。しかし、サイコロの個数を増やすことで、確率の分布は大きく変化する。 n 個のサイコロが同じ目を出す確率を計算するために、式 1 を用いる。

$$P = \frac{s}{s^n} = \frac{1}{s^{n-1}} \quad (1)$$

- n 個のサイコロのすべての組み合わせは s^n 通り。

- 全てのサイコロが同じ出目になる場合の数は s 通り（全て 1、全て 2、…、全て s ）。
- よって、確率は次のようになります：

$$P = \frac{\text{一致する場合の数}}{\text{全体の組み合わせ数}} = \frac{s}{s^n} = \frac{1}{s^{n-1}}$$

本実験では 7 個の 6 面サイコロをシミュレーションするため、すべてのサイコロが同じ目を示す確率は式 2 に示す通り、 2.143×10^{-5} となる。

$$P = \frac{1}{6^{7-1}} = \frac{1}{6^6} = \frac{1}{46656} \approx 2.143 \times 10^{-5} \quad (2)$$

2.1.2 課題 2: 半径 7 の円の面積

本シミュレーションでは、2 次元平面上に半径 7 の円を描く。乱数により生成された位置 (x, y) のプロットを平面上に配置し、そのプロットが円の内部に存在する理論的確率を計算し、シミュレーション結果と比較する。

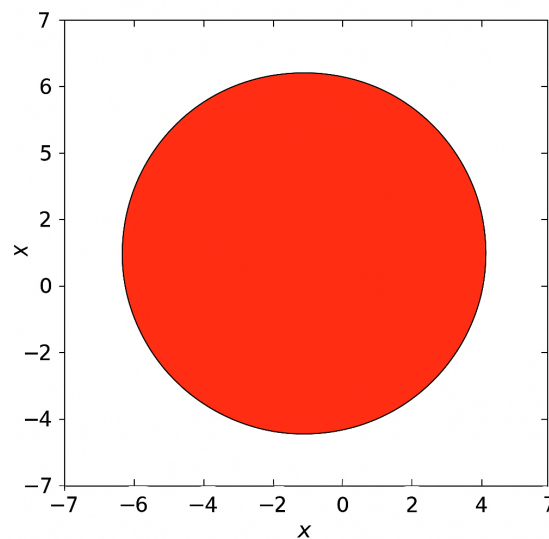


図 2: 半径 7 の円の面積

乱数によって生成されたプロットが円の内部に入る理論確率の計算は、式 3 に示すように、円の面積を全体の面積で割るだけである。本シミュレーションを簡略化するため、プロットの座標の上限および下限を -7 から 7 に制限し、面積を 14×14 に設定する。

$$P = \frac{\text{円の面積}}{\text{領域の面積}} = \frac{\pi r^2}{A} \quad (3)$$

この場合, 全体の領域の一辺が 14 であることが分かっているため, 面積は $A = 14 \times 14 = 196$ と計算できる. 一方, 円の半径を 7 に設定しているため, 円の面積は $\pi \times r \times r = \pi \times 7 \times 7 = 49\pi$ と計算できる. これら二つの値を式 3 に代入することで, 式 4 を得り, 乱数によって生成されたプロットが円の内部に入る確率は約 0.785 となる.

$$P = \frac{49\pi}{196} = \frac{\pi}{4} \approx 0.785 \quad (4)$$

2.1.3 課題 3: 異身長男女を一行に並べる

本実験では, 身長異なる 10 名 (男性 5 名, 女性 5 名) をランダムに一行に並べ, 各女子の左側に, 自身よりも背の高い男子が 1 人以上いる並びとなる確率をシミュレーションにより求める.

各人の身長の関係は以下のように定義されており, すべての男性および女性は明確な順序で区別されている:

$$M_1 > W_1 > M_2 > W_2 > M_3 > W_3 > M_4 > W_4 > M_5 > W_5 \quad (5)$$

この関係に従って, 10 名を一行にランダムに並べる. その上で, すべての女性について, 「自身より背の高い男性が左側に少なくとも 1 人存在する」かどうかを調べる. この条件をすべての女性が満たす並びが, 全体の中でどの程度の確率で現れるかを, シミュレーションにより統計的に推定する.

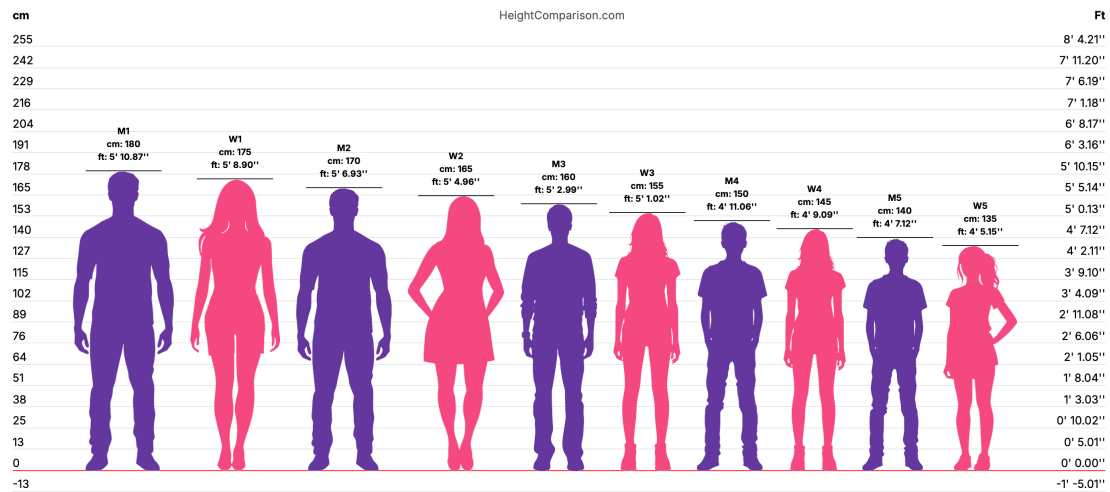


図 3: 男女 10 名の身長関係と識別

しかしながら、本問題の条件により、理論的な割合や期待値の計算は非常に複雑になる。先に述べた通り、列内の各女性に対して、少なくとも一人はその女性より身長の高い男性が左側に存在しなければならない。そのため、本シミュレーションでは 10 人の並びの全ての順列を総当たりで検証し、条件を満たすかどうかを確認する手法を用いる。

n 個の対象を一行に並べる場合の総順列数は、式 6 により計算できる。

$$n! = n \times (n-1) \times (n-2) \times \cdots \times 2 \times 1 \quad (6)$$

今回のように、身長異なる 10 人（あるいは 10 個の対象）を一行に並べる場合、式 6 を用いることで、順列の総数は

$$10! = 10 \times 9 \times 8 \times 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1 = 3,628,800$$

通りであることが分かる。

次に、Python のコードを用いて、先に定義した条件に対して行のすべての順列を検証した。各順列をチェックするために使用した Python コードはコード 1 に示す。

コード 1: 順列を確認するコード

```
def is_valid(permutation):
    for i, person in enumerate(permutation):
        if person.startswith('W'):
            w_height = height_rank[person]
            left = permutation[:i]
            found_taller_man = any(p.startswith('M')
                                   and
                                   height_rank[p] < w_height for p in left)
            if not found_taller_man:
                return False
    return True
```

コードを各順列を確認する結果は表 1 に表されている。ブルートフォース法を用いて、条件を満たす順列の数が 893025 であることを確認した。有効な順列の数を全順列の数で割ることにより、理論的確率または期待値が 0.2461 となることが得られた。

表 1: 身長条件シミュレーションにおける順列結果

項目	値
総順列数	3,628,800
有効な順列数	893,025
確率 (割合)	0.2461

2.1.4 課題 4: ジャンケンの割合

ジャンケンシミュレーションは、伝統的なジャンケンに得点制度を導入したシミュレーションである。提示された条件に従い、特定の手による勝敗によって得られる点数（本稿ではドル単位）が異なる。完全なルールは表 2 に示す。

表 2: じゃんけんの勝利手ごとの報酬（単位：ドル）

あなたの手 \ 相手の手	グー	パー	チョキ
グー (G)	0	-10	+10
パー (P)	+30	0	-30
チョキ (T)	-20	+20	0

ジャンケンに設定されたルールが複雑であるため、最も高い得点を得る各手（グー、チョキ、パー）の理論的な確率や期待される比率を正確に求めることはほぼ不可能である。しかし、式 7 を用いることで、その比率を近似的に求めることができる。

$$E = \sum_{i \in \{R, P, S\}} \sum_{j \in \{R, P, S\}} P_{\text{you}}(i) \cdot P_{\text{opp}}(j) \cdot \text{reward}(i \text{ vs } j) \quad \text{ここで } E \text{ は期待利益となる} \quad (7)$$

本初回のシミュレーションでは、相手の手は偏りなくランダムに選ばれる。そのため、各手の比率は正確に $\frac{1}{3}$ となる。先に表 2 で示した各手の勝敗に伴う得失を踏まえ、式 7 に代入することで、相手が偏りない場合に最も最適な勝率または戦略の比率を算出できる。

$$\begin{aligned}
E(r, p, s) &= r \cdot \left(\frac{1}{3} \cdot 0 + \frac{1}{3} \cdot (-30) + \frac{1}{3} \cdot (+10) \right) \\
&\quad + p \cdot \left(\frac{1}{3} \cdot (+30) + \frac{1}{3} \cdot 0 + \frac{1}{3} \cdot (-20) \right) \\
&\quad + s \cdot \left(\frac{1}{3} \cdot (-10) + \frac{1}{3} \cdot (+20) + \frac{1}{3} \cdot 0 \right) \\
&= r \cdot \left(\frac{-30 + 10}{3} \right) + p \cdot \left(\frac{30 - 20}{3} \right) + s \cdot \left(\frac{-10 + 20}{3} \right) \\
&= r \cdot \left(\frac{-20}{3} \right) + p \cdot \left(\frac{10}{3} \right) + s \cdot \left(\frac{10}{3} \right) \\
&= \frac{1}{3} (-20r + 10p + 10s)
\end{aligned} \tag{8}$$

最大の利益 (E) を得るためには, $r + p + s = 1$ かつ $r, p, s \geq 0$ を満たす条件下で, $E(r, p, s)$ を最大化する必要がある. $r + p + s = 1$ であるため, 式 8 における s を $1 - r - p$ に置き換えることができ, これにより計算を式 9 の形に簡略化できる.

$$\begin{aligned}
E(r, p) &= \frac{1}{3} (-20r + 10p + 10(1 - r - p)) \\
&= \frac{1}{3} (-20r + 10p + 10 - 10r - 10p) \\
&= \frac{1}{3} (-30r + 10)
\end{aligned} \tag{9}$$

式 9 の内容を検討すると, r が増加するにつれて $E(r, p)$ の値は減少する. また, $E(r, p)$ を最大化する必要がある, $r \geq 0$ であることから, $E(r, p)$ は $r = 0$ のときに最大値をとる. 最後に, r を 0 に置き換えると, p と s の合計比率が 1 である条件のもとで, 式 10 に示すように期待利益は約 3.33 となる. この場合, p と s の任意の組み合わせ (例えば 0.2 と 0.8, 0.6 と 0.4 など) でも最大利益を得られることになる.

$$E(0, p) = \frac{1}{3} (0 + 10) = 3.33 \tag{10}$$

2.1.5 課題 5: ビンゴのシミュレーション

本シミュレーションでは, 表 3 に示した条件に従い, ランダムに生成されたビンゴカードを作成する.

表 3: ビンゴカードの列と対応する数値範囲

列	数値範囲
1 列目 (B)	1 ~ 15
2 列目 (I)	16 ~ 30
3 列目 (N)	31 ~ 45
4 列目 (G)	46 ~ 60
5 列目 (O)	61 ~ 75

また、従来のビンゴのルールでは、フルビンゴを達成するために 5 列すべてを完成させる必要がある（各列につき 1 つ）。しかし本シミュレーションでは、1 列の完成をフルビンゴとみなす。完成した列は、ビンゴカード上の任意の位置における横、縦、または斜めのいずれかとなる。また、ビンゴカードの中央はフリースペースとされ、ゲーム開始時点で既にこのマスは開いているものとする。

ビンゴゲームにおいて、以下の 3 つの異なる条件がシミュレーションされた。

1. 7 回以内で当たる確率をシミュレーション

本シミュレーションは、7 回のコール以内当たる確率を求めることを目的とする。

2. 最短回数 (4 回) で当たりとなる確率をシミュレーション

本シミュレーションは、最短ターン数である 4 回以内（ビンゴカード中央が既に開いていることを考慮）に当たる確率を算出することを目的とする。

3. 当たりとなる最も遅いゲーム回数とそのような状態の発生確率をシミュレーション

本シミュレーションは、最も遅い（コール数が最大となる）タイミングで当たる確率を算出することを目的とする。ビンゴカードには、表 3 に示した通り、75 の候補番号のうち 24 個（ 5×5 のマスから中央の 1 マスを除く）が記載されている。これは、ビンゴカード上の番号に当たらずに呼ばれることが可能な番号が $75 - 24 = 51$ 個存在することを意味する。ビンゴカードにならない番号をすべて呼んだ後でも、表 4 に示すように、ビンゴカード上で列が完成しないまま呼べる番号がさらに 19 個存在する。

表 4: 完全列または表を当たらないビンゴカード

B	I	N	G	O
O	X	X	X	X
X	X	O	X	X
X	X	X	O	X
X	O	X	X	X
X	X	X	X	O

つまり, 列や行が完成しないまま呼べる番号の総数は $51 + 19 = 70$ となる. 言い換えれば, 最悪の場合でも 71 回目のコールで必ずビンゴが達成されることになる.

これらのシミュレーションにおける正確な理論的確率や期待値を計算することは極めて複雑であり, 単一の数値を導出するための単一の方程式は存在しない.

しかし, 表 3 に示したビンゴカード作成のルールを考慮すると, 式 11 に示す組み合わせの公式を用いて, 可能な組み合わせ数を計算することができる.

$${}^n\text{Cr}(n, r) = \frac{n!}{r!(n-r)!} \quad (11)$$

$$\text{B 列: } {}^{15}C_5 = 3003$$

$$\text{I 列: } {}^{15}C_5 = 3003$$

$$\text{N 列: } {}^{15}C_4 = 1365 \quad (\text{中心の FREE スペースのため 4 つ選ぶ})$$

$$\text{G 列: } {}^{15}C_5 = 3003$$

$$\text{O 列: } {}^{15}C_5 = 3003$$

(12)

$$\text{合計のカード数: } ({}^{15}C_5)^4 \times {}^{15}C_4 = 3003^4 \times 1365$$

$$\begin{aligned} \text{数値計算: } 3003^4 &\approx (3.003 \times 10^3)^4 = 3.003^4 \times 10^{12} \approx 81.2 \times 10^{12} \\ &= 8.12 \times 10^{13} \end{aligned}$$

$$8.12 \times 10^{13} \times 1365 \approx 1.1 \times 10^{17}$$

したがって, 式 12 の計算により, ビンゴカードの総数は約 1.1×10^{17} . この総数で条件を満たすカードを計算することは本手法は多大な計算資源を要する. それにより, 各シミュレーション条件の確率を近似するためには, シミュレーションを用いる方が望ましい.

2.2 シミュレーションの役割

現実には、与えられた条件に対する理論的確率や期待値の導出は、ルールの変更やバイアスの導入によって困難になる場合がある。例えば、節 2.1.1 および 2.2.2 で示したように、6 面ダイスや単純な 2 次元円を用いた場合の理論的確率の算出は、シミュレーション自体で考慮すべき要素が少ないため容易である。しかし、ダイスに偏りがあり、特定の面が他の面よりも出やすいといった条件を導入した場合、理論的にはその偏りの度合いが正確に分かっていれば、バイアスを計算し理論値を求めることは可能である。とはいえ、実際には複数の予期しない要因が結果に影響を及ぼす可能性があり、このような解析ははるかに困難となる。[4]

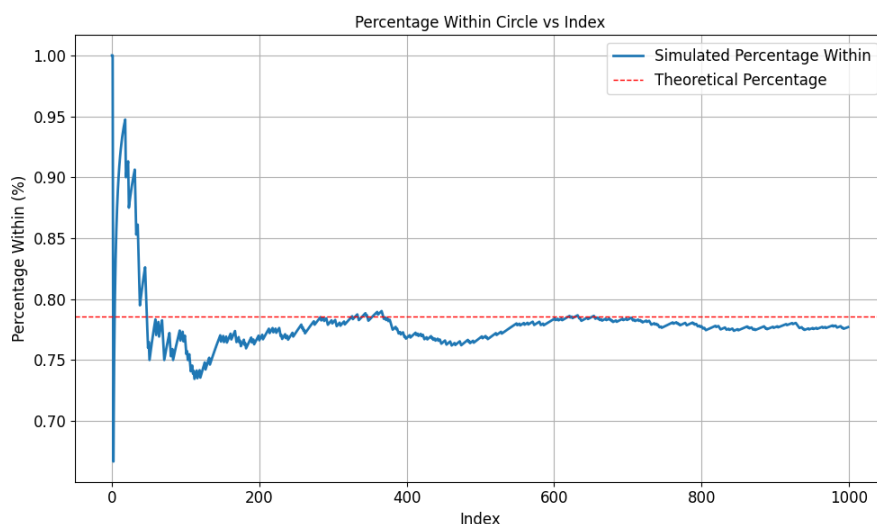


図 4: 出現率が収束する例

このような場合において、理論的確率を直接計算するのではなく、シミュレーションを用いることがはるかに有効な代替手段となる。シミュレーションでは、条件に従って同じ実験を何度も繰り返し実行し、有効な結果のみを記録すればよい。シミュレーションの回数を十分に重ねることで出現率は次第に安定し、図 4 に示すように、当該条件における出現確率のおおよその傾向を把握することが可能となる。

2.3 モンテカルロシミュレーション

モンテカルロシミュレーションとは、複雑な条件における確率を得るために繰り返しのサンプリングを用いる計算アルゴリズムの一種である。節 2.2 で述べたように、条件によっては理論的な確率を正確に計算することが計算コストの面で非常に高く、多大な時間を要する場合がある。モンテカルロシミュレーションを用いることで、すべての順列を網羅的に列挙することなく、不確実な事象の発生確率を推定

することが可能となる．たとえば，節 2.1.5 で定義された制約下におけるビンゴのシミュレーションでは，ビンゴカードの可能な順列数が 1.1×10^7 以上存在するため，全順列を評価するには数日から場合によっては数年を要する可能性がある．このような場合にモンテカルロシミュレーションを用いることで，当該事象の確率を現実的な計算時間内で推定することができる．[2]

3 実験方法

本節では，節 2.1 で述べた実験において使用される各種手法および関数について説明する．

3.1 ハードウェアと環境

ハードウェアとしては Apple シリコンのプロセッサに基づいて実験を行う．

機種名 Apple Macbook Pro (M3, 2023 年モデル)

プロセッサ Apple M3 チップ (8 コア：高性能コア 4 + 高効率コア 4)

GPU 10 コア統合型 GPU

メモリ 16GB ユニファイドメモリ

ストレージ 1TB SSD

OS macOS Sequoia 15.5

アーキテクチャ ARM64 (Apple シリコン)

また，作成した実験プログラムは C 言語で書き，GCC コンパイラでコンパイルされる．

```
Apple clang version 17.0.0 (clang-1700.0.13.5)
```

```
Target: arm64-apple-darwin24.5.0
```

```
Thread model: posix
```

コンパイルを効率化するため，GCC コンパイラを Make と同時に利用する．

```
GNU Make 3.81
```

```
Copyright (C) 2006 Free Software Foundation, Inc.
```

```
This is free software; see the source for copying conditions.
```

```
There is NO warranty; not even for MERCHANTABILITY or
```

```
FITNESS FOR A PARTICULAR PURPOSE.
```

```
This program built for i386-apple-darwin11.3.0
```

3.2 ランダム関数

C言語において、乱数値を生成する最も一般的な方法の一つは、`stdlib.h` ライブラリが提供する `rand()` 関数である。`rand()` 関数は通常、乱数の種（シード）を設定するための `srand()` 関数と組み合わせて用いられる。一般に「乱数」とは、次にどの値が出るか予測できないことを意味するが、広く使われている `rand()` 関数の実装は厳密には真の乱数ではなく、疑似乱数である。先に述べたように、`rand()` 関数は `srand()` 関数と組み合わせて使用され、乱数生成の開始点となる種を設定する。この種は、疑似乱数生成の計算基準となる開始値である。この開始値は式 13 におく LCG (Linear Congruential Generator) であり、予め定められた定 x 数群を用いて次の疑似乱数を生成するために用いられる。

$$X_{n+1} = (aX_n + c) \bmod m \quad (13)$$

この式は `rand()` 関数内に実装されており、コード 2 に類似した形で表される。

コード 2: `rand()` 関数の簡単版

```
static unsigned long next = 1;

int rand(void) {
    next = next * 1103515245 + 12345;
    return (unsigned int)(next / 65536) % 32768;
}

void srand(unsigned int seed) {
    next = seed;
}
```

一定の式を用いているため、乱数生成は開始種が分かっているならば、実際には生成される値を予測可能である。これは、`stdlib.h` ライブラリが提供する一般的な `rand()` 関数はエントロピーが低く、 m 回の出力後には必ず生成される値が繰り返されることを意味する.[1]

モンテカルロシミュレーションにおいて、この制約はシミュレーションの精度を著しく低下させる。なぜなら、可能なすべてのケースや順列を十分に一般化するために、シミュレーションを可能な限り多く繰り返す必要があるからである.[6]

この制約を回避するために、すべてのシミュレーションは、`stdlib.h` ライブラリが提供する `arc4random` の拡張である `arc4random_uniform` 関数を用いて実行す

る。rand 関数とは異なり、arc4random は LCG（線形合同法）を使用せず、より安全な CSPRNG（暗号的に安全な疑似乱数生成器）を採用している。rand と arc4random はいずれも真の乱数ではなく疑似乱数であるが、arc4random はデバイスのハードウェアから得られるエントロピー値を乱数の種として利用する点が異なる。一方で、一般的な rand 関数はエポック時刻を基に初期シードを設定しているため、実行時刻が正確にわかれば、次に生成される乱数を「予測」できる可能性がある。[7]

arc4random 関数を実行すると、デバイスは /dev/urandom から疑似乱数バイト列を読み取る。この urandom ファイルは、デバイスのカーネルが管理する内部エントロピープールから生成される。エントロピープールは、ネットワークタイミグ、キーボードやマウスの入出力、割り込みタイミグなど、ハードウェアに基づくさまざまな値からシードされる。/dev/urandom から読み取った内容の一例をコード 3 に示す。

コード 3: urandom の内容のを読み取り結果例

```
head -c 16 /dev/urandom | xxd
00000000:
e690 1de3 6b5c 6c27 7684 fc82 6cfa a9a3
....k\l'v...l...
head -c 16 /dev/urandom | xxd
00000000:
c15c a4e2 c72e 8d0d 5494 4f21 874b d0b3
.\.....T.O!.K..
head -c 16 /dev/urandom | xxd
00000000:
3198 d4b2 7c21 5e2b 41a6 7a94 8012 f0ab
1...|!^+A.z.....
```

これらのランダムな値は、ChaCha20 アルゴリズムに入力される。ChaCha20 は、Daniel J. Bernstein によって 2008 年に設計されたストリーム暗号であり、実用的な暗号解析に対して高い耐性を持つ。現代の暗号技術において広く用いられており、OpenSSH や TLS（Transport Layer Security）、各種オペレーティングシステムなど、業界標準の技術においても利用されている。[5]

arc4random は、低レイヤのハードウェア層から生成されるエントロピー値を元にシードが設定されるという特性を持つ。このため、arc4random は通常の rand 関数と比較して、より「ランダム」に近い値を生成することが可能となる。

3.3 モジュロによるランダム関数のバイアス

ランダム値ジェネレータ内部には、値をユーザに返す前に行われる最終的な操作が存在する。この操作はモジュロチェックと呼ばれ、生成された値に対してあらかじめ設定された最大値で剰余演算を行うことで、値がその範囲を超えないように制限される。しかし、この手法はランダム値を特定の範囲内で生成したい場合に大きな問題を引き起こす。[3] 例えば、ランダム値ジェネレータが符号なし 8 ビット整数の最大値である 255 まで出力可能だとする。このとき、`arc4random` 関数に対して最大値を 120 と指定した場合、120 を超えるすべての数値は先頭から再び数えられるように変換される。これにより、予期せぬバイアスが発生し、特定の値が他の値よりも頻繁に出力される傾向を持つようになる。

`arc4random` と `arc4random_uniform` の違いを適切に評価するために、C 言語を用いて小規模なシミュレーションを実施した。`arc4random` は上下限の設定を直接サポートしないため、コード 4 に示すように再実装を行った。

コード 4: 再実装した `arc4random` 関数

```
int rand_in_range(int min, int max){
    return arc4random() % (max - min + 1) + min;
}
```

一方、`arc4random_uniform` は下限をサポートしないため、コード 5 に示すようにこちらも再実装を行った。

コード 5: 再実装した `arc4random_uniform` 関数

```
int rand_uniform_in_range(int min, int max){
    return arc4random_uniform(max - min + 1) + min;
}
```

両関数を下限 1, 上限 100 で 10 万回繰り返し実行した結果は、図 5 に示すように類似している。

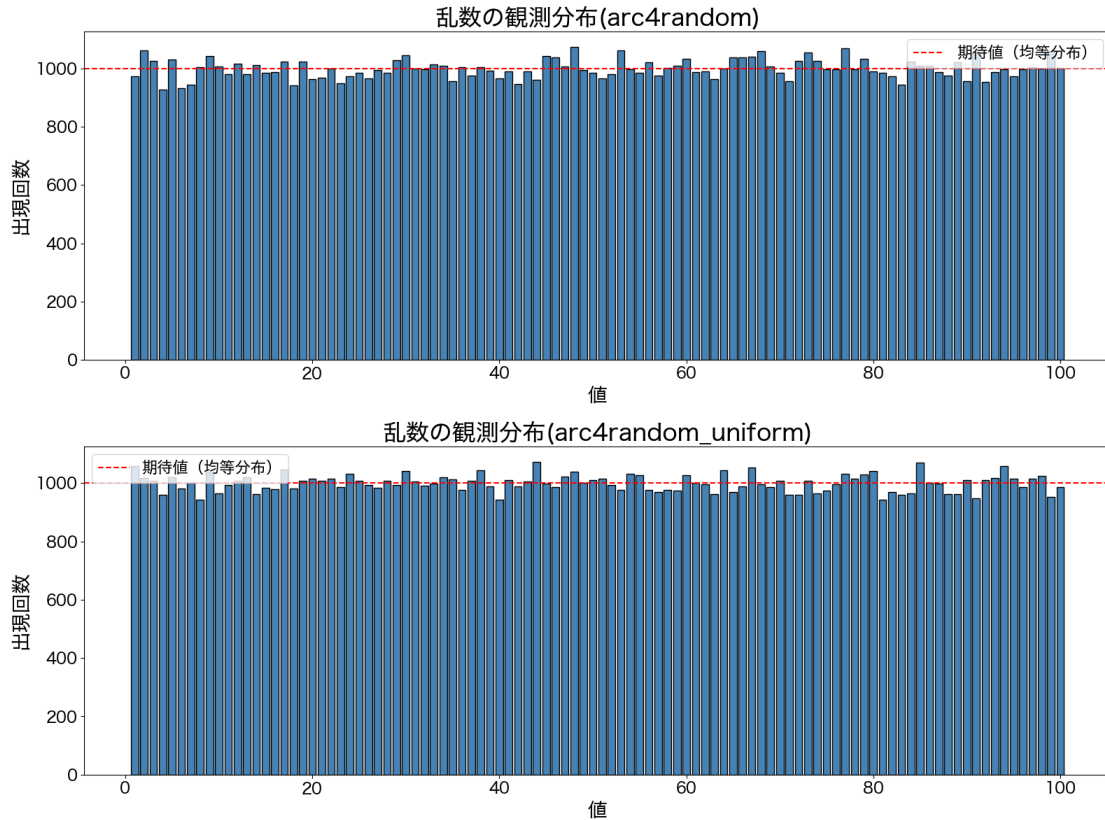


図 5: arc4random と arc4random_general によって生成された乱数の比較 (下限 1, 上限 100)

グラフをそのまま比較した場合、2つの関数間に有意な差があるとは言い難い。適切な比較を行うために、両手法の p 値およびカイ二乗値を算出する。

表 5 の結果から、モンテカルロシミュレーションに必要な高いエントロピーを arc4random_uniform の結果において確認できる。エントロピーが高いほど、生成される値はよりランダムであることを示す。一方で、カイ二乗値が低いことは、この結果が一様分布に近いことを意味する。

両関数とも p 値に基づくバイアスは有意ではないものの、arc4random_uniform のエントロピースコアが高いことにより、シミュレーションデータに含まれる潜在的なノイズが除去される。以上の理由から、本実験ではすべてのシミュレーションに arc4random_uniform 関数を用いる。

表 5: カイ二乗検定の結果

メソッド	カイ二乗統計量	P 値
<code>arc4random</code>	103.030000000	0.370758315
<code>arc4random_uniform</code>	83.980000000	0.859645089

3.4 実行数

前節 2.3 で述べたように, モンテカルロシミュレーションでは理論値に近い結果を得るために大量の反復回数が必要となる. サイコロのような単純なシミュレーションでは 1000 回の反復で十分な場合もあるが, より複雑なルールセットや稀な事象を扱うシミュレーションでは, 正確な結果を得るために 100 万回以上の反復が必要となることがある. 適切な反復回数の設定は困難であり, 多くの試行錯誤を要する.

4 実験結果

参考文献

- [1] Wolfgang Hörmann and Gerhard Derflinger. A portable uniform random number generator well suited for the rejection method. WorkingPaper 3, Institut für Statistik und Mathematik, Abt. f. Angewandte Statistik u. Datenverarbeitung, WU Vienna University of Economics and Business, 1992.
- [2] IBM. What is monte carlo simulation?, 2021. Accessed: 2025-07-20.
- [3] Daniel Lemire. Fast random integer generation in an interval. *ACM Transactions on Modeling and Computer Simulation (TOMACS)*, 29(1):1–12, 2019.
- [4] Tim P. Morris, Ian R. White, and Michael J. Crowther. Using simulation studies to evaluate statistical methods. *Statistics in Medicine*, 38(11):2074–2102, January 2019.
- [5] OpenBSD Project and BSD Manual Pages. *arc4random(3): High-quality CSPRNG functions*, 2025. Manual page, OpenBSD (and FreeBSD/macOS).
- [6] Giovanni Ossola and Alan D. Sokal. Systematic errors due to linear congruential random-number generators with the swendsen-wang algorithm: A warning. *Physical Review E*, 70(2), August 2004.

- [7] The OpenBSD Project. arc4random — cryptographic random number generator. <https://man.openbsd.org/arc4random>, 2024. Accessed: 2025-07-20.